PROJECT – 3

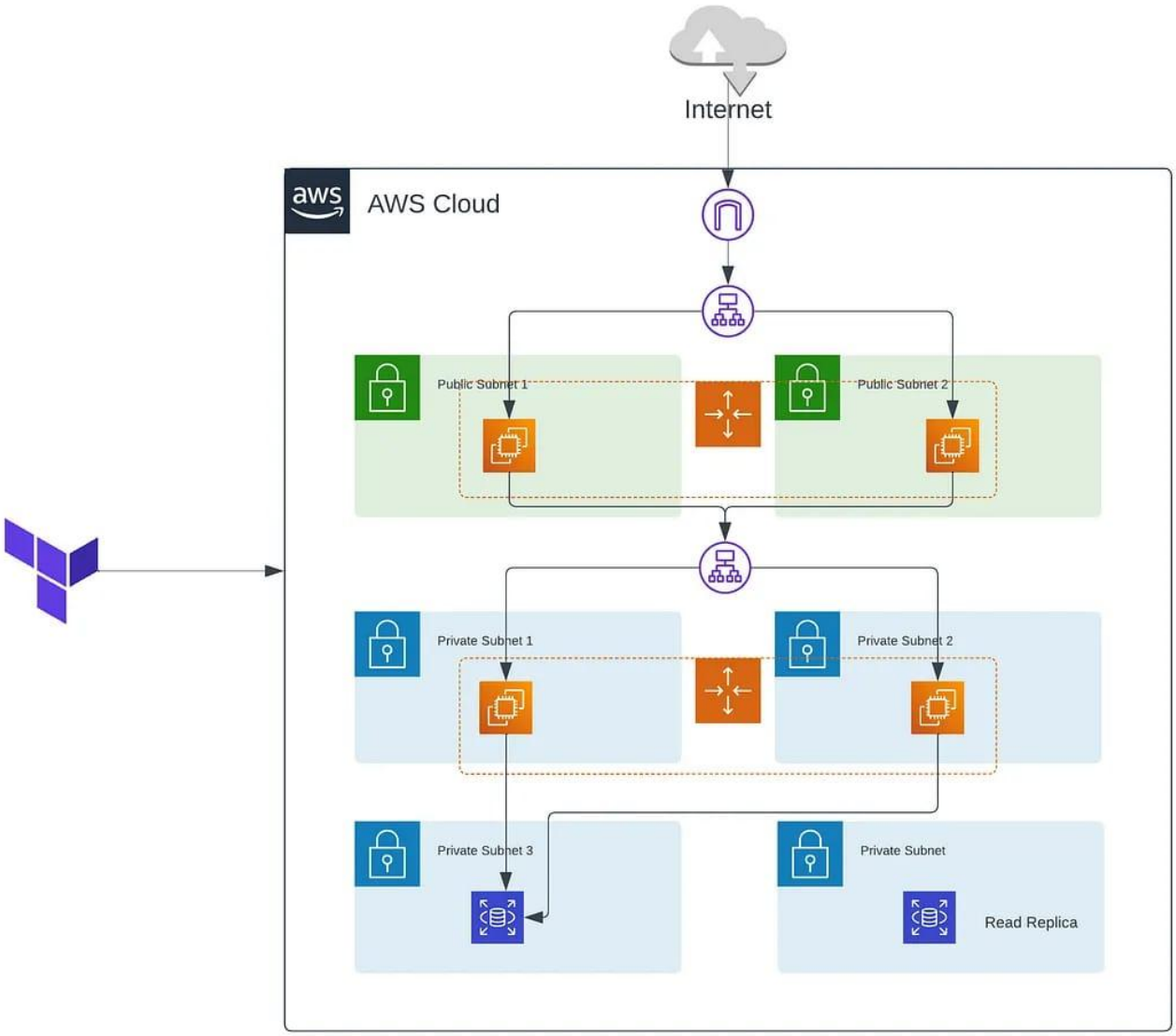
3-TIER ARCHITECTURE USING TERRAFORM

Presentation By : M.Karthik Reddy

Batch : 141

Institute : Vcube Software Solutions

3-TIER ARCHITECTURE USING TERRAFORM



We Consider the Following:

Iac_Tool: Terraform

Cloud_Platform: AWS

Services_Used: a. VPC

b. EC2

c. RDS

Introduction: 3-Tier Architecture Using Terraform

This project demonstrates the automation and deployment of a complete 3-Tier Architecture on AWS using Terraform as the Infrastructure-as-Code (IaC) tool. A 3-tier architecture separates an application into three independent layers—Presentation, Application, and Database—to achieve improved scalability, security, and maintainability. Terraform automates the provisioning of all cloud resources through declarative configuration, ensuring consistent, repeatable, and version-controlled infrastructure deployments.

At the core of this project is the AWS Provider in Terraform, which interacts directly with AWS services to create networking, compute, and database components. The project begins with a custom VPC containing multiple public and private subnets across different availability zones. Public subnets host internet-facing EC2 instances behind a Public Application Load Balancer, forming the Presentation Tier. These instances receive and distribute incoming user traffic securely over the internet.

The Application Tier is deployed inside private subnets for security. EC2 instances in this layer are isolated from the internet and accessible only through a Private/Internal Load Balancer that receives traffic from the web tier. Security Groups enforce strict communication rules between tiers, ensuring controlled access and protecting internal resources.

The Database Tier uses Amazon RDS MySQL, deployed entirely in private subnets. Automated backups and a Read Replica enhance availability and performance. Only the application servers can access the database, ensuring strong data security and compliance.

Terraform's AWS Provider automates all components—VPC, subnets, route tables, NAT Gateway, Internet Gateway, EC2 instances, load balancers, and RDS. This results in a fully automated, scalable, and secure cloud environment that reflects real-world production architectures.

Overall, the project showcases the practical use of Terraform to build a reliable 3-tier infrastructure on AWS, combining automation, cloud architecture best practices, and strong security principles into a cohesive, production-ready deployment.

PROCESS OF 3-Tier Architecture Using Terraform

STEP – 1: CREATE TERRAFORM_PROJECT DIRECTORY

- Open The Git Bash
- Run Mkdir Terraform_Project Directory
- Change the working directory Desktop to Terraform_Project (Ex: Cd Terraform_Project)

STEP – 2: CREATE PROVIDER.TF FILE

- Enter vi provider.tf
- Write the aws cloud provider code

```
provider "aws" {
region = "us-west-1"
profile = "default"
}
```

Fig-1: Provider block code

STEP – 3: CREATE RESOURCE.TF FILE

- Enter vi Resource.tf
- Write the aws cloud resources code

a) Create VPC Resource Code

- Label the VPC Name as "Project-vpc"
- Give IPv4 CIDR as "10.0.0.0/16"

```
# VPC Resource
# ---------------------------------------------------
resource "aws_vpc" "main" {
 cidr_block = "10.0.0.0/16"
 enable_dns_hostnames = true
 enable_dns_support  = true

 tags = {
  Name = "Project-vpc"
 }
}
```

Fig -2: vpc resource code

b) Create Subnets Resource Code

- Create 6 subnets : 2 public, 4 private
- Label the subnets as “Public & Private-subnets”
- Give IPv4 CIDR as “10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24, 10.0.3.0/24, 10.0.4.0/24, 10.0.5.0/24, 10.0.6.0/24”

```
# Public Subnets
# ----------------------------------------------
resource "aws_subnet" "public_1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone = "us-west-1a"

  tags = { Name = "public-subnet-1" }
}

resource "aws_subnet" "public_2" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  map_public_ip_on_launch = true
  availability_zone = "us-west-1c"

  tags = { Name = "public-subnet-2" }
}
```

```
# Private Subnets
# ----------------------------------------------------
resource "aws_subnet" "private_1" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.3.0/24"
  availability_zone = "us-west-1a"

  tags = { Name = "private-subnet-1" }
}

resource "aws_subnet" "private_2" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.4.0/24"
  availability_zone = "us-west-1c"

  tags = { Name = "private-subnet-2" }
}

resource "aws_subnet" "private_3" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.5.0/24"
  availability_zone = "us-west-1a"

  tags = { Name = "private-subnet-3" }
}

resource "aws_subnet" "private_4" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.6.0/24"
  availability_zone = "us-west-1c"

  tags = { Name = "private-subnet-4" }
}
```

Fig-3: Public & Private Subnets Resource code

c) Create Route Tables Code

- Create 2 Route Tables : 1 public, 1 private
- Label the subnets as “Public-rt & Private-rt”

```
# Route Tables
# ---------------------------------------------
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = { Name = "public-rt" }
}

resource "aws_route_table_association" "public_1_assoc" {
  subnet_id      = aws_subnet.public_1.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "public_2_assoc" {
  subnet_id      = aws_subnet.public_2.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }

  tags = { Name = "private-rt" }
}
```

Fig-3: Public & Private-rt resource code

d) Create Associate Route tables to subnet code

- Associate route tables to subnets
- Public-rt associate to public subnets
- Private-rt associate to private subnets

```
resource "aws_route_table_association" "private_1_assoc" {
  subnet_id      = aws_subnet.private_1.id
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_route_table_association" "private_2_assoc" {
  subnet_id      = aws_subnet.private_2.id
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_route_table_association" "private_3_assoc" {
  subnet_id      = aws_subnet.private_3.id
  route_table_id = aws_route_table.private_rt.id
}
```

Fig-4: Public & Private route tables associate to subnets code

e) Create Internet Gateway Code

- Create a internet gateway
- Attach Igw to vpc

```
# Internet Gateway
# ------------------------------------------------
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Project-igw"
  }
}
```

Fig-5: Internet gateway Resource code

f) Create Nat Gateway and Elastic Ip Code

- Create a nat gateway
- Allocate eip to nat gateway

```
# NAT Gateway + EIP
# ------------------------------------------------
resource "aws_eip" "nat_eip" {
  domain = "vpc"
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public_1.id

  tags = { Name = "Project-nat-gateway" }
}
```

Fig-6: Nat Gateway resource code

g) Create Public Ec2 Instances

- Create 2 public instances named as “public-ec2-1, public-ec2-2”
- Lanch in different availability zones(1a,1c)

```
# EC2 in Public Subnets
# ------------------------------
resource "aws_instance" "public_ec2_1" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.public_1.id
  security_groups = [aws_security_group.public_sg.id]

  tags = { Name = "public-ec2-1" }
}


resource "aws_instance" "public_ec2_2" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.public_2.id
  security_groups = [aws_security_group.public_sg.id]

  tags = { Name = "public-ec2-2" }
}
```

Fig-7: Public ec2 resource code

h) Create Private Ec2 Instances

- Create 2 private instances named as “private-ec2-1, private-ec2-2”
- Lanch in different availability zones(1a,1c)

```
# Private EC2
# ------------------------------
resource "aws_instance" "private_ec2_1" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.private_1.id
  security_groups = [aws_security_group.private_sg.id]

  tags = { Name = "private-ec2-1" }
}

resource "aws_instance" "private_ec2_2" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.private_2.id
  security_groups = [aws_security_group.private_sg.id]

  tags = { Name = "private-ec2-2" }
}
```

Fig-8: Private ec2 resource code

i) Create Public Application Load Balancer

- Create a public application load balancer and target group
- After that add to listener

```
# Public Application Load Balancer
# ------------------------------

resource "aws_lb" "public_alb" {
  name               = "public-app-lb"
  internal           = false
  load_balancer_type = "application"
  subnets            = [
    aws_subnet.public_1.id,
    aws_subnet.public_2.id
  ]
  security_groups    = [aws_security_group.public_sg.id]

  tags = {
    Name = "public-alb"
  }
}

resource "aws_lb_target_group" "public_tg" {
  name     = "public-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id

  health_check {
    path = "/"
    protocol = "HTTP"
    port = "80"
  }
}

resource "aws_lb_listener" "public_listener" {
  load_balancer_arn = aws_lb.public_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.public_tg.arn
  }
}
```

Fig-9: Public application load balancer resource code

j) Attach Public Ec2 to Load Balancer

- Attach 2 public ec2 to load balancer

```
# Attach Public EC2 to Public ALB
# -------------------------------
resource "aws_lb_target_group_attachment" "public_ec2_attach_1" {
  target_group_arn = aws_lb_target_group.public_tg.arn
  target_id        = aws_instance.public_ec2_1.id
  port             = 80
}

resource "aws_lb_target_group_attachment" "public_ec2_attach_2" {
  target_group_arn = aws_lb_target_group.public_tg.arn
  target_id        = aws_instance.public_ec2_2.id
  port             = 80
}
```

Fig-10: Attach public ec2 to load balancer resource code

k) Create Private Application Load Balancer

- Create a private application load balancer and target group
- After that add to listener

```
# Private Application Load Balancer
# -------------------------------

resource "aws_lb" "private_alb" {
  name               = "private-app-lb"
  internal           = true
  load_balancer_type = "application"
  subnets            = [
    aws_subnet.private_1.id,
    aws_subnet.private_2.id
  ]
  security_groups    = [aws_security_group.private_sg.id]

  tags = {
    Name = "private-alb"
  }
}

resource "aws_lb_target_group" "private_tg" {
  name     = "private-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id

  health_check {
    path = "/"
    protocol = "HTTP"
    port = "80"
  }

  tags = {
    Name = "private-tg"
  }
}

resource "aws_lb_listener" "private_listener" {
  load_balancer_arn = aws_lb.private_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.private_tg.arn
  }
}
```

Fig-11: Public application load balancer resource code

l) Attach Private Ec2 to Private application Load Balancer

- Attach 2 private ec2 to load balancer

```
# Attach Private EC2 to Pravite ALB
# -------------------------------
resource "aws_lb_target_group_attachment" "private_ec2_attach_1" {
  target_group_arn = aws_lb_target_group.private_tg.arn
  target_id        = aws_instance.private_ec2_1.id
  port             = 80
}

resource "aws_lb_target_group_attachment" "private_ec2_attach_2" {
  target_group_arn = aws_lb_target_group.private_tg.arn
  target_id        = aws_instance.private_ec2_2.id
  port             = 80
}
```

Fig-12: Attach private ec2 to load balancer resource code

m) Create Database Subnets Group

- Create a Subnets Group private-3 &4

```
# RDS Master
# -------------------------------
resource "aws_db_subnet_group" "db_subnets" {
  name       = "db-subnet-group"
  subnet_ids = [
    aws_subnet.private_3.id,
    aws_subnet.private_4.id
  ]

  tags = { Name = "db-subnet-group" }
}
```

Fig-13: Database subnets group resource code

n) Create RDS Database

- Create Database
- Choose Engine as: MySQL
- In Credentials Management choose myself
- Give new password to it

```
resource "aws_db_instance" "db" {
  identifier           = "main-db"
  engine               = "mysql"
  instance_class       = "db.t3.micro"
  username             = "admin"
  password             = "Admin12345"
  allocated_storage    = 20
  backup_retention_period = 1
  skip_final_snapshot  = true
  publicly_accessible  = false
  db_subnet_group_name = aws_db_subnet_group.db_subnets.name
  vpc_security_group_ids = [aws_security_group.db_sg.id]
}
```

Fig-14: Create RDS Database resource code

o) Create Read Replica

- Add Read Replica to Database

```
resource "aws_db_instance" "read_replica" {
  identifier           = "read-replica"
  replicate_source_db  = aws_db_instance.db.arn
  instance_class       = "db.t3.micro"
  publicly_accessible  = false
  skip_final_snapshot  = true
  db_subnet_group_name = aws_db_subnet_group.db_subnets.name
  vpc_security_group_ids = [aws_security_group.db_sg.id]
}
```

Fig-15: Create read replica resource code

STEP – 4: CREATE DATASOURCE BLOCK.TF FILE

- Enter vi datasourceblock.tf
- Write a code for ec2 instance Ubuntu ami

```
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  owners = ["099720109477"]
}
```

Fig-16: Create a datasource block

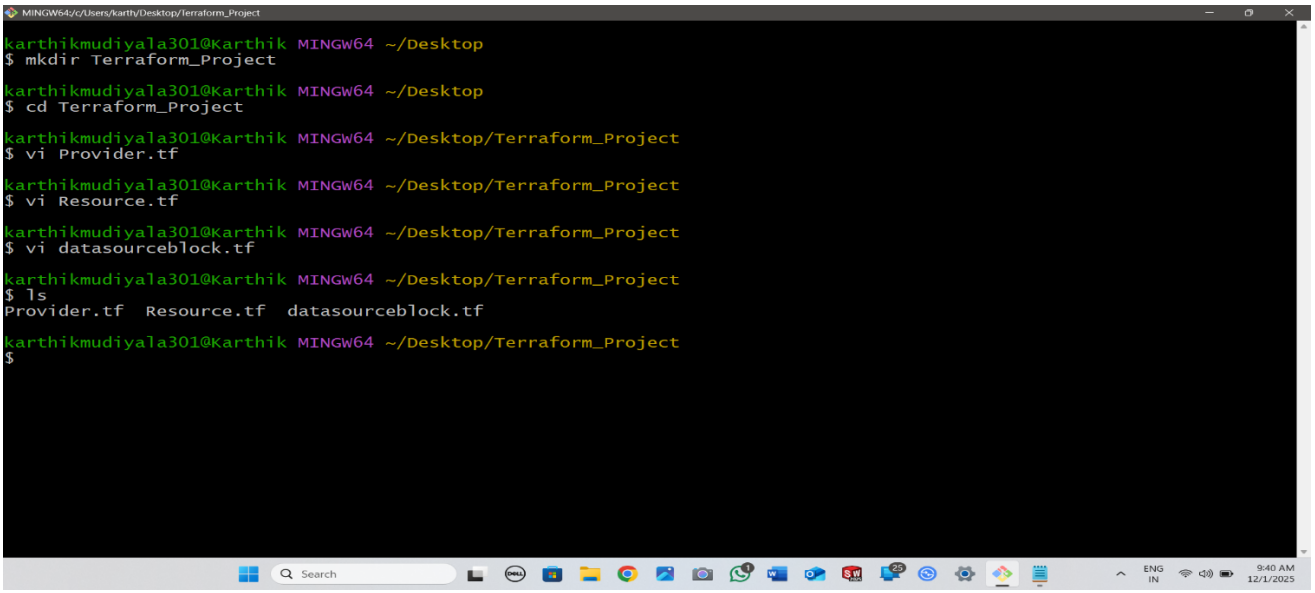
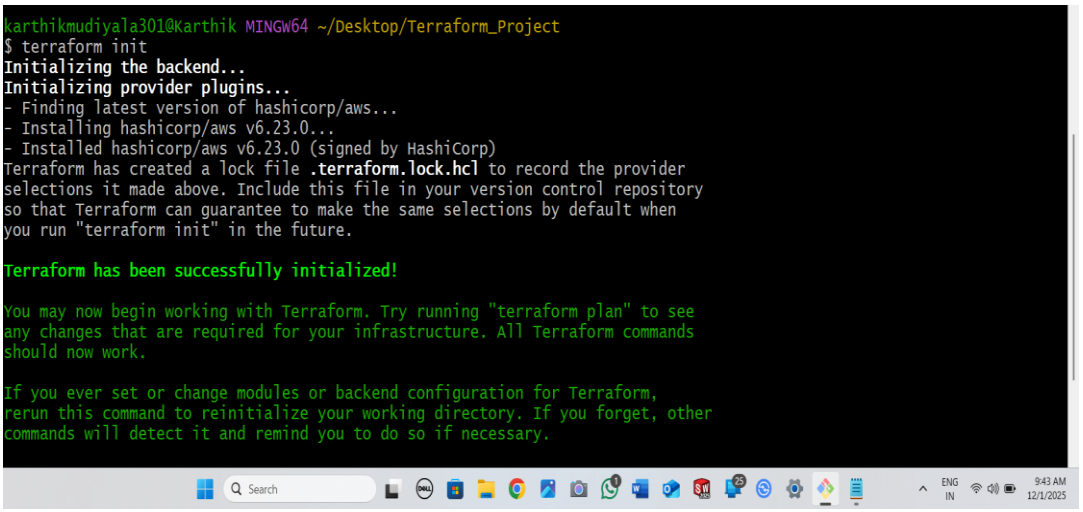


Fig-17: Check all Blocks in Terraform_Project Directory

STEP – 5: RUN THE TERRAFORM COMMANDS

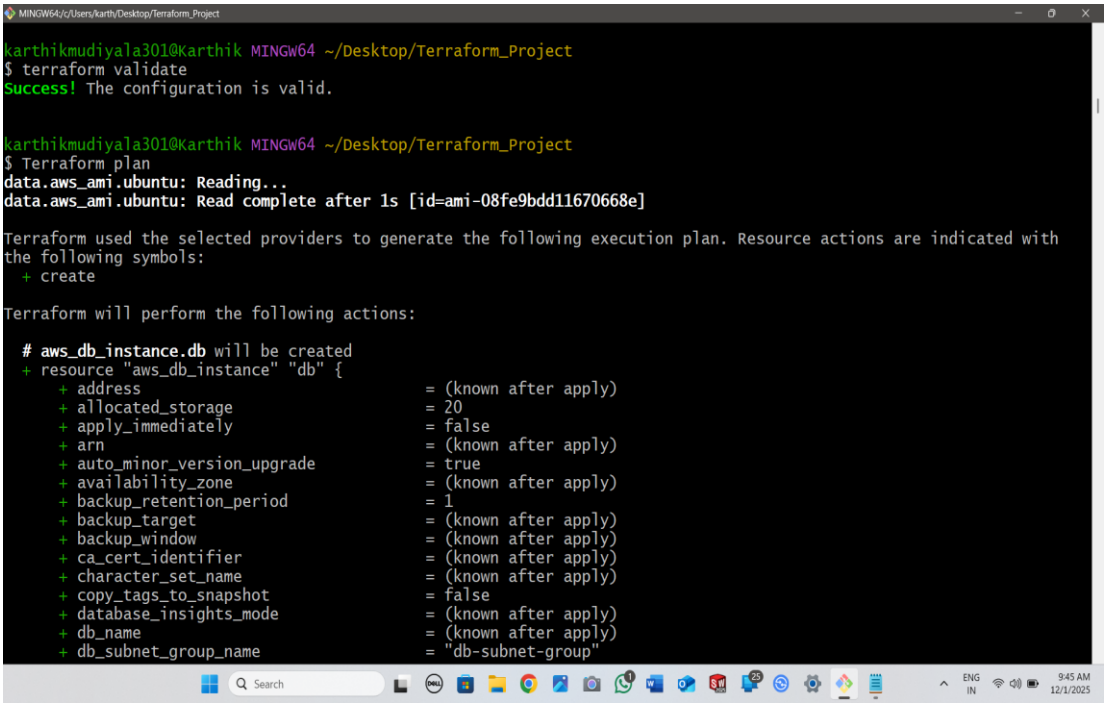
a) Terraform init



```
karthikmudiyala301@Karthik MINGW64 ~/Desktop/Terraform_Project
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.23.0...
- Installed hashicorp/aws v6.23.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Fig-18: Run Terraform init Command

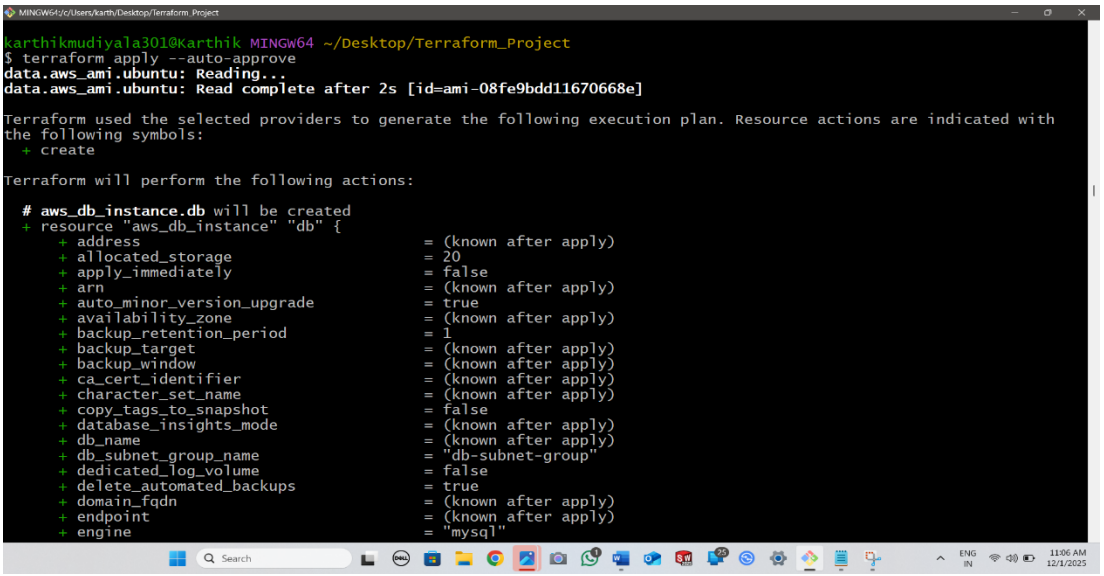
b) Terraform Validate
c) Terraform Plan



```
MINGW64:/c/Users/karth/Desktop/Terraform_Project

karthikmudiyala301@Karthik MINGW64 ~/Desktop/Terraform_Project
$ terraform validate
Success! The configuration is valid.


karthikmudiyala301@Karthik MINGW64 ~/Desktop/Terraform_Project
$ Terraform plan
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 1s [id=ami-08fe9bdd11670668e]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
  + create

Terraform will perform the following actions:

  # aws_db_instance.db will be created
  + resource "aws_db_instance" "db" {
      + address                               = (known after apply)
      + allocated_storage                     = 20
      + apply_immediately                     = false
      + arn                                   = (known after apply)
      + auto_minor_version_upgrade            = true
      + availability_zone                     = (known after apply)
      + backup_retention_period               = 1
      + backup_target                         = (known after apply)
      + backup_window                         = (known after apply)
      + ca_cert_identifier                    = (known after apply)
      + character_set_name                    = (known after apply)
      + copy_tags_to_snapshot                 = false
      + database_insights_mode                = (known after apply)
      + db_name                               = (known after apply)
      + db_subnet_group_name                  = "db-subnet-group"
```

Fig-19: Run Terraform validate and Terraform plan Commands

d) Terraform Apply



```
karthikmudiyala301@Karthik MINGW64 ~/Desktop/Terraform_Project
$ terraform apply --auto-approve
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 2s [id=ami-08fe9bdd11670668e]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
  + create

Terraform will perform the following actions:

  # aws_db_instance.db will be created
  + resource "aws_db_instance" "db" {
      + address                               = (known after apply)
      + allocated_storage                     = 20
      + apply_immediately                     = false
      + arn                                   = (known after apply)
      + auto_minor_version_upgrade            = true
      + availability_zone                     = (known after apply)
      + backup_retention_period               = 1
      + backup_target                         = (known after apply)
      + backup_window                         = (known after apply)
      + ca_cert_identifier                    = (known after apply)
      + character_set_name                    = (known after apply)
      + copy_tags_to_snapshot                 = false
      + database_insights_mode                = (known after apply)
      + db_name                               = (known after apply)
      + db_subnet_group_name                  = "db-subnet-group"
      + dedicated_log_volume                  = false
      + delete_automated_backups              = true
      + domain_fqdn                           = (known after apply)
      + endpoint                              = (known after apply)
      + engine                                = "mysql"
```

Fig-20: Run Terraform apply Command

STEP – 6: AUTOMATICALLY CREATE AWS RESOURCES IN CONSOLE USING TERRAFORM COMMANDS

- After the successfully execute the terraform apply command automatically create the required resources in aws console.

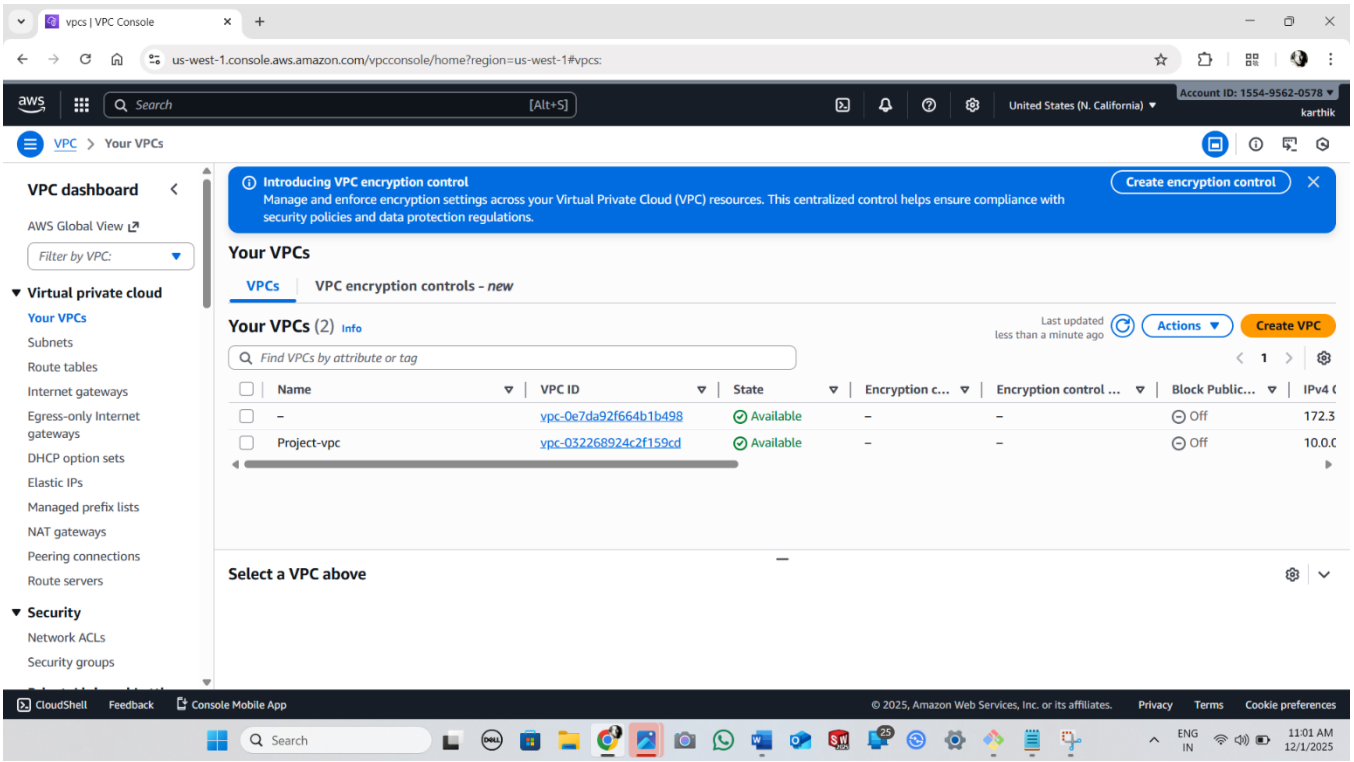


Fig-21: Vpc

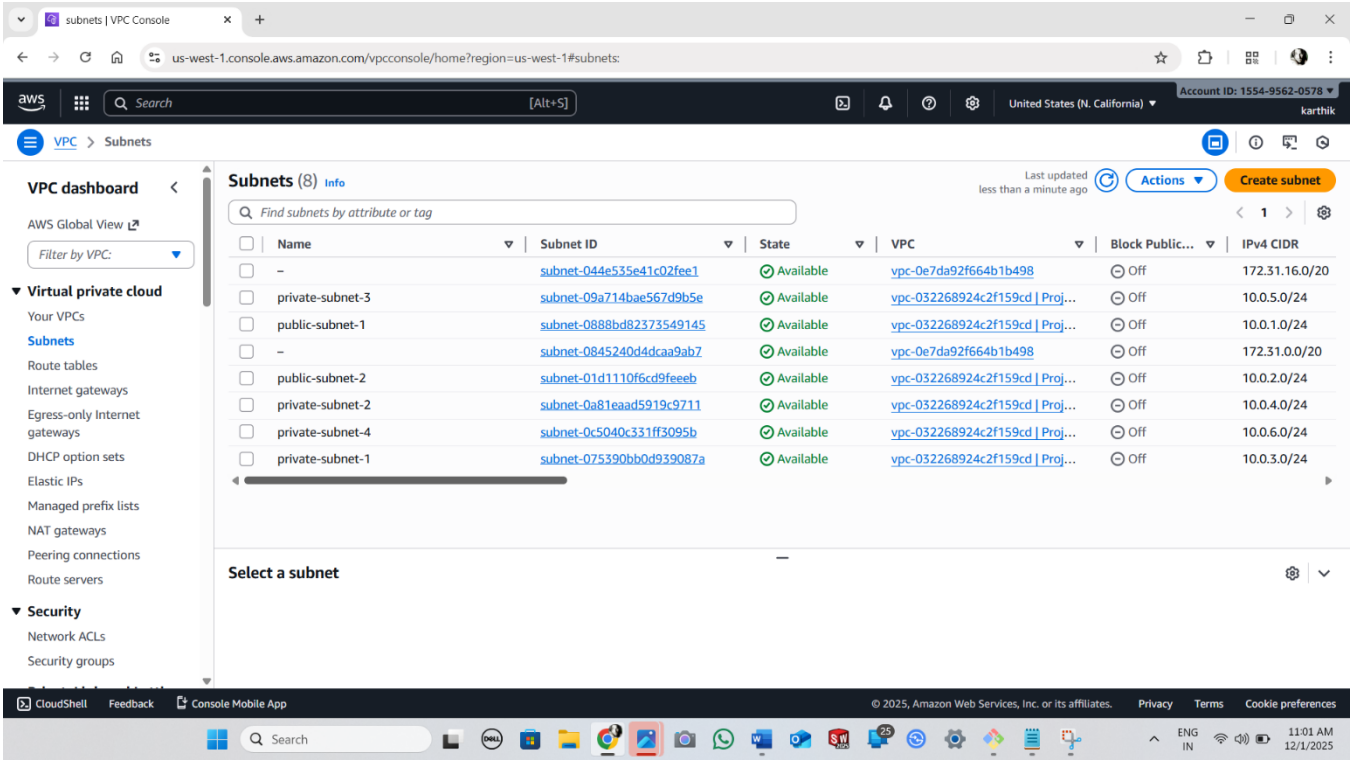


Fig-22: Public & Private subnets

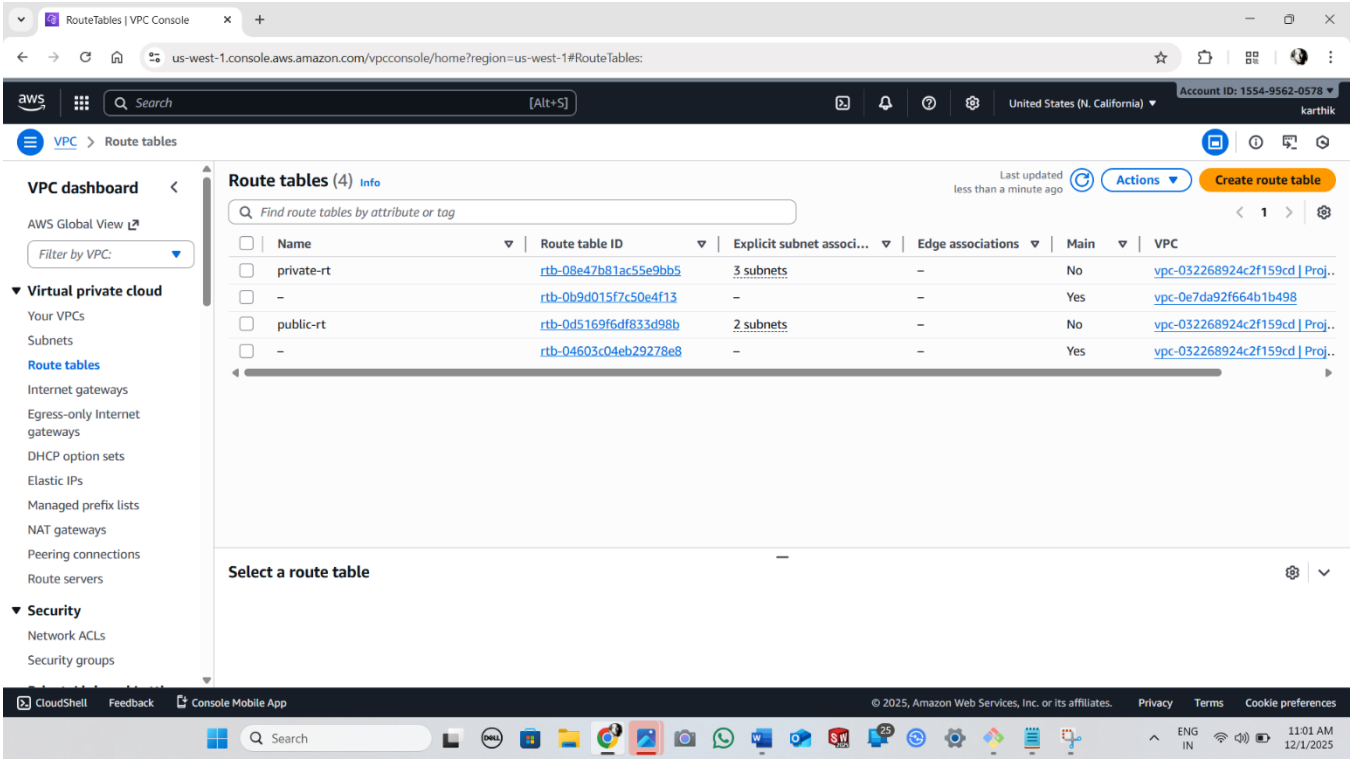


Fig-23: Public & Private Route tables

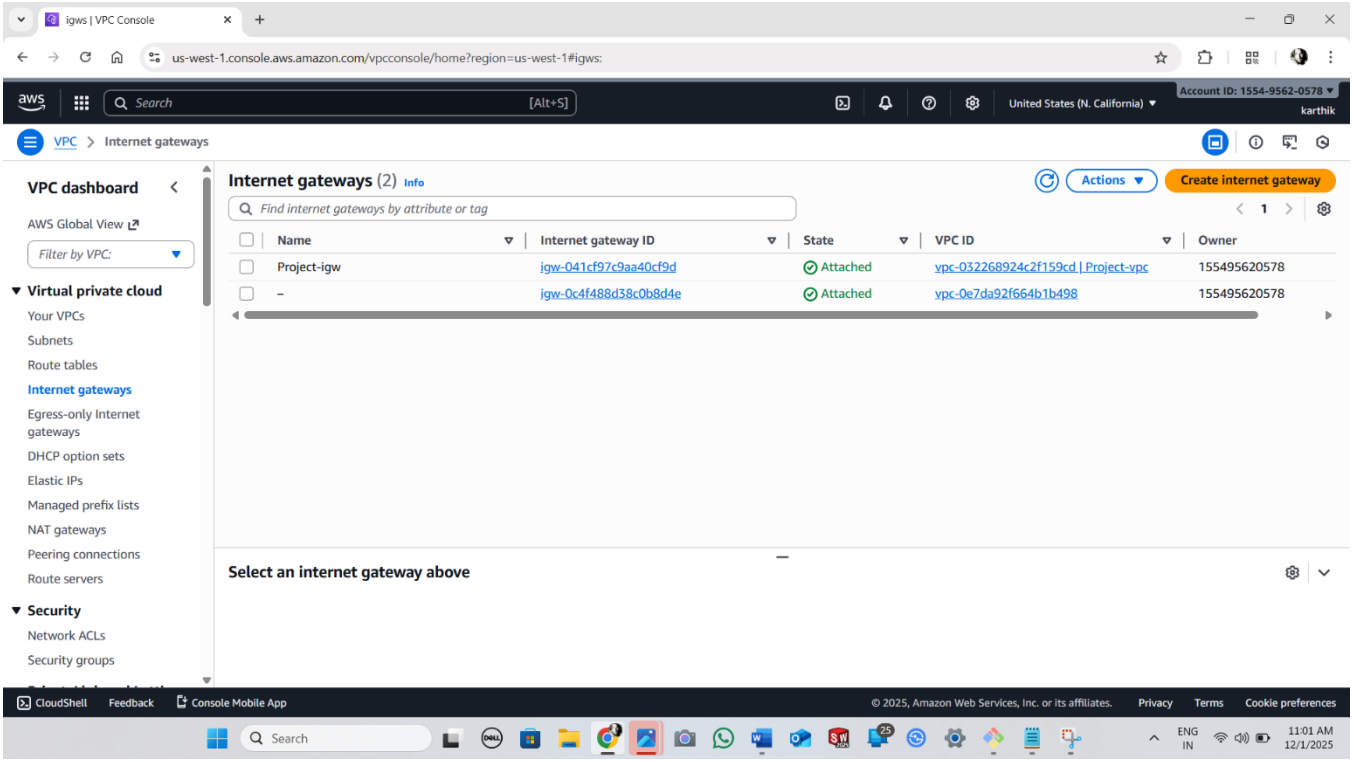


Fig-24: Internet gateway

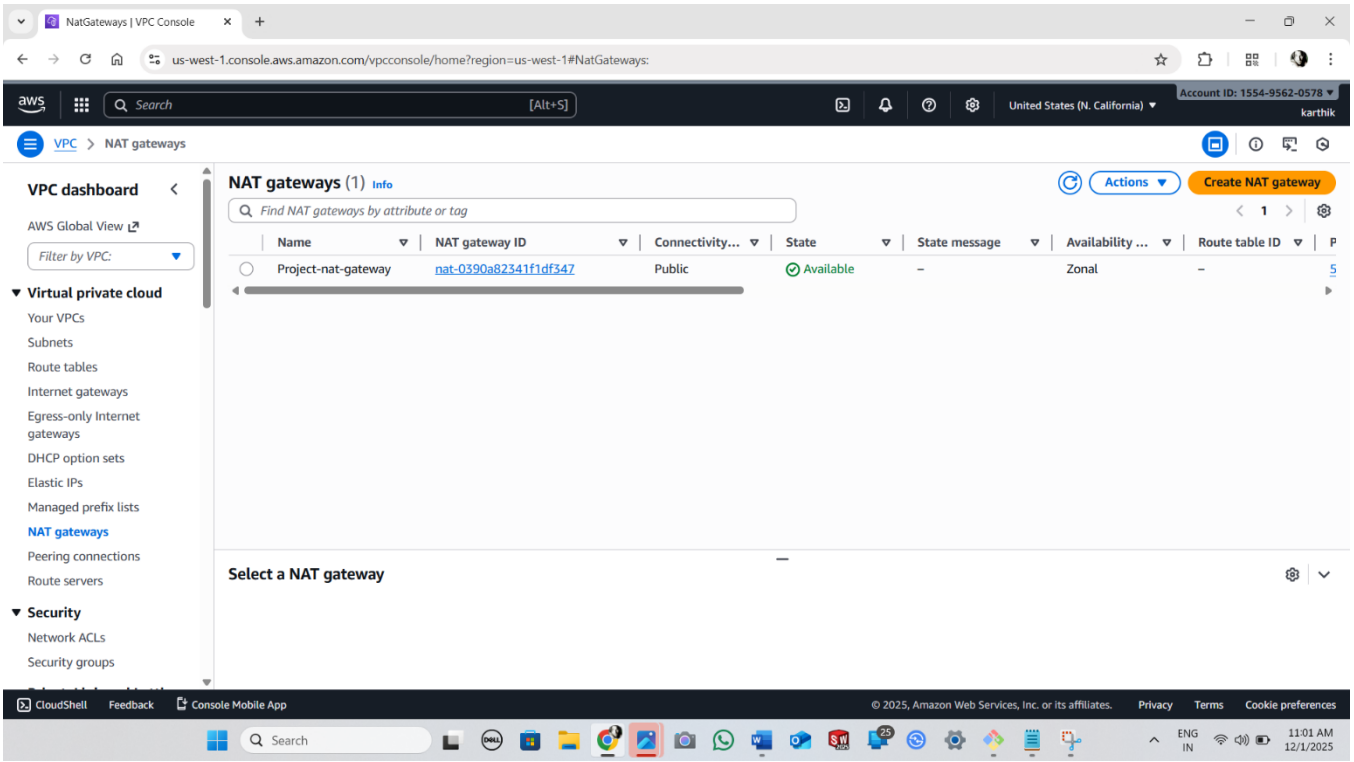


Fig-25: NAT Gateway

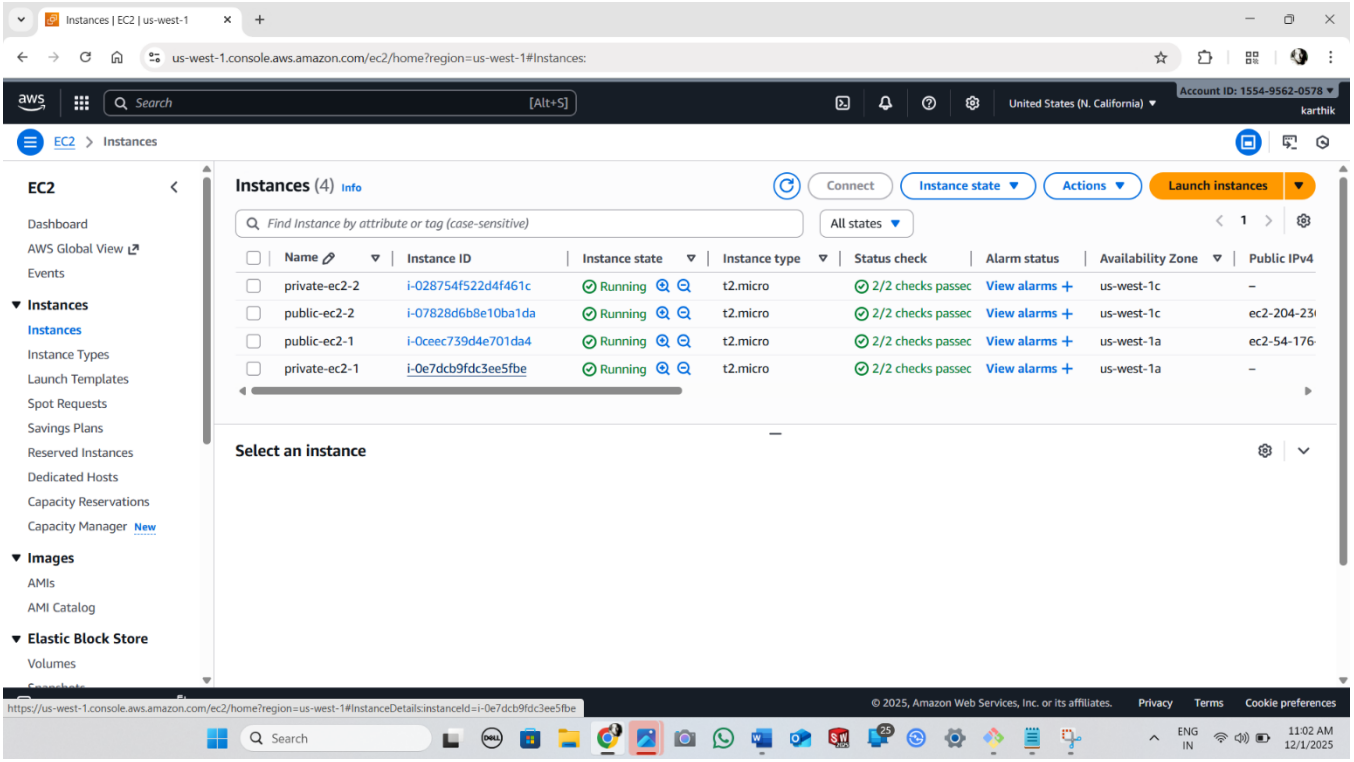


Fig-26: Ec2 Instances

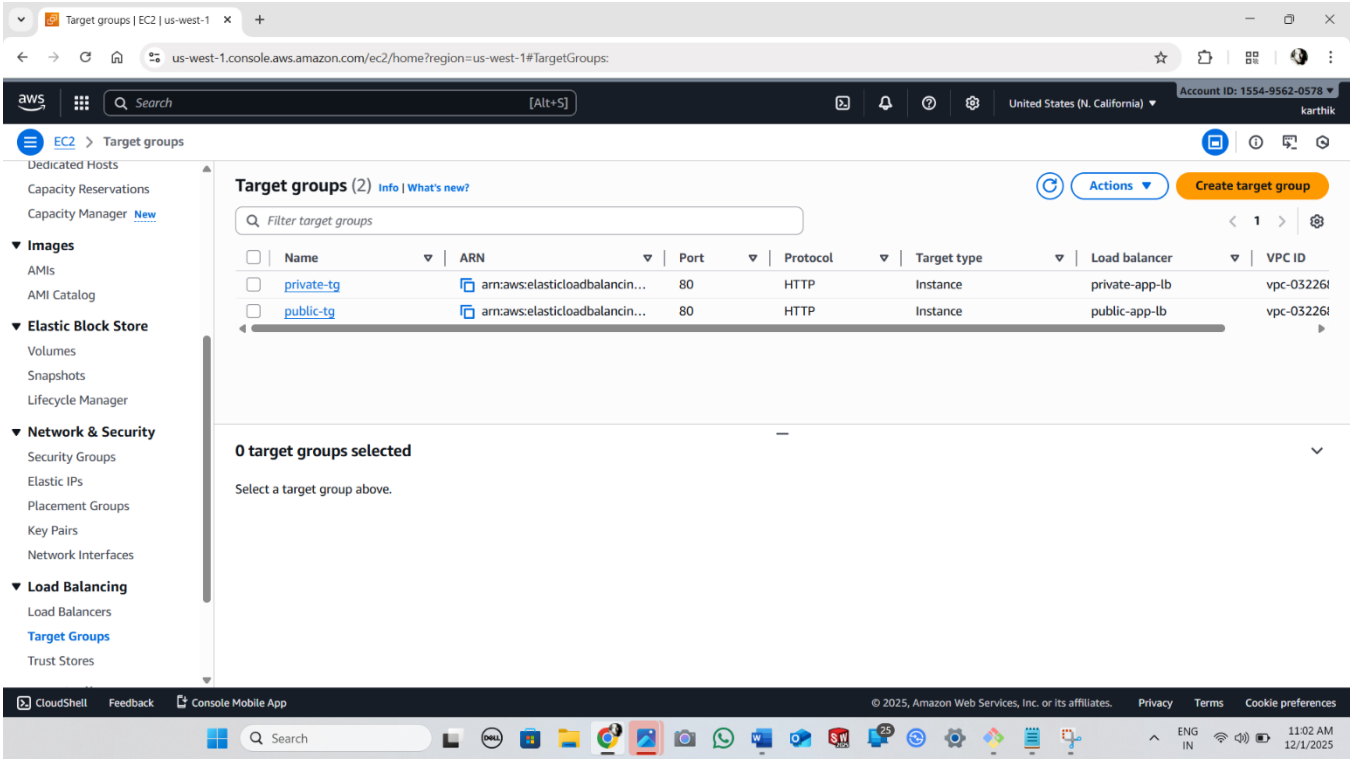


Fig-27: Target groups

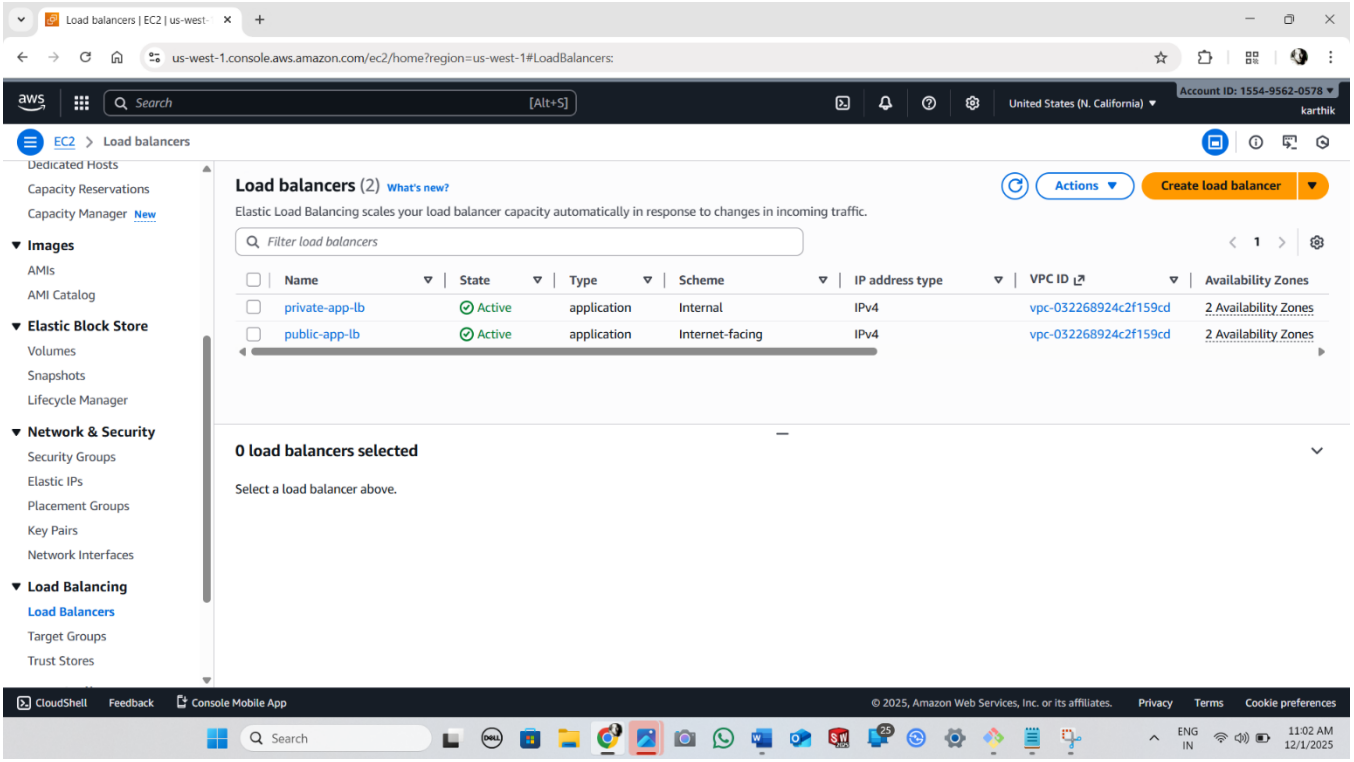


Fig-28: Application Load Balancers

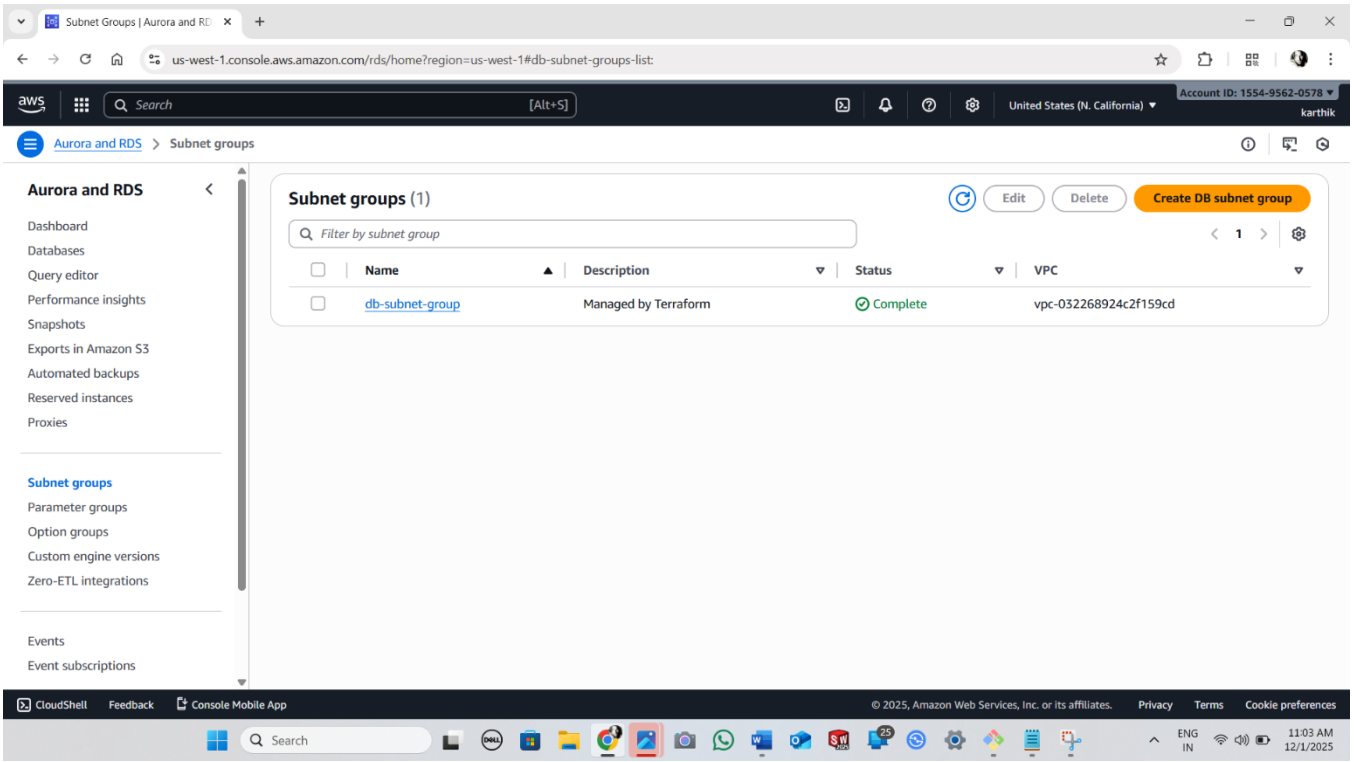


Fig-29: Database Subnet Group

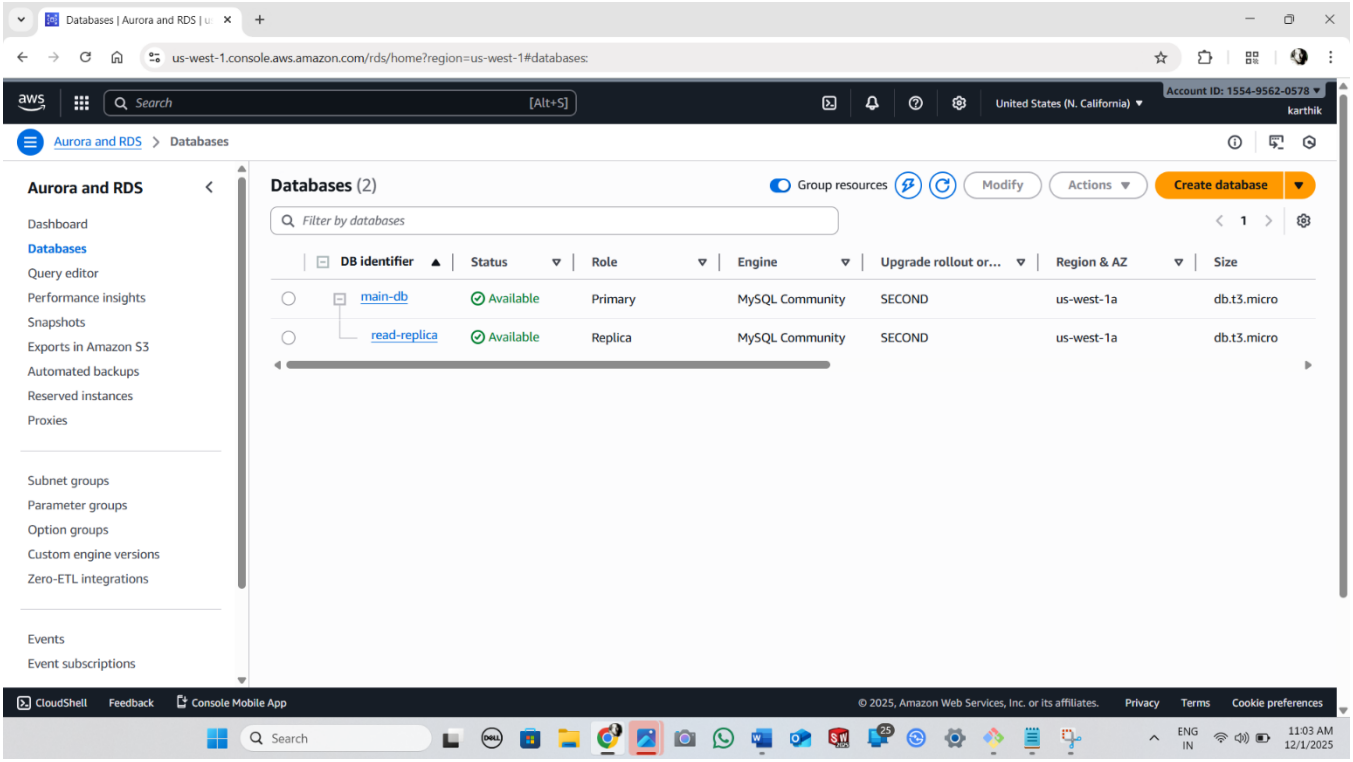


Fig-30: Read Replica

- If You want to see the aws resources connection by using the terraform using **"Terraform graph command"**.

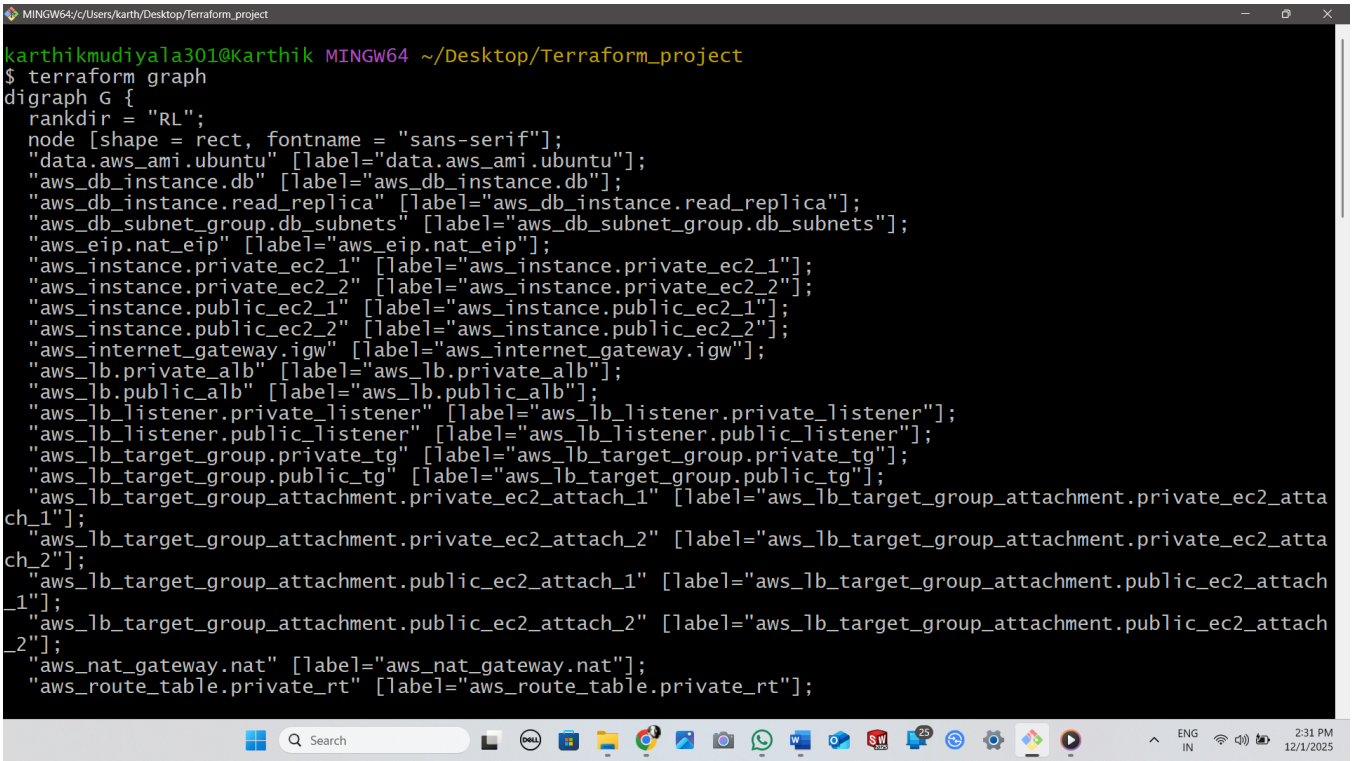


Fig-31: Generate a graph using the command Terraform Graph

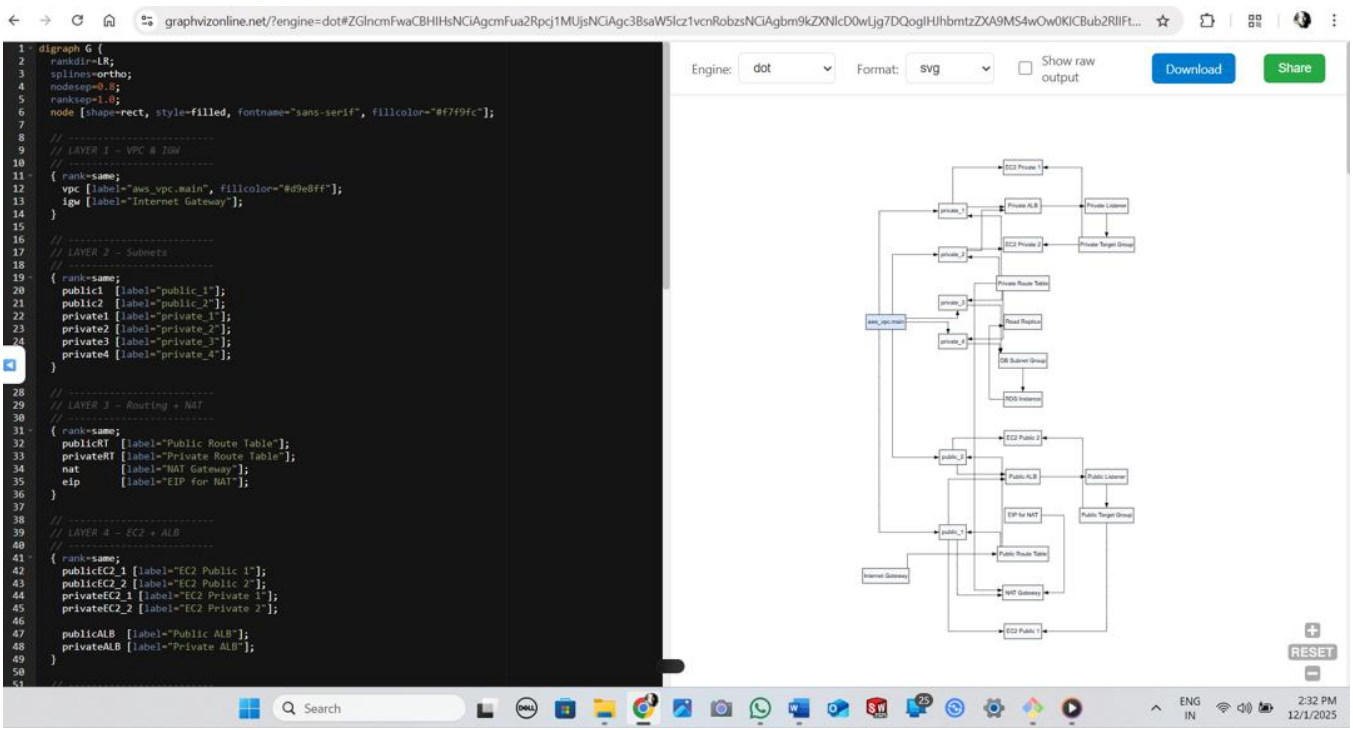


Fig-32: Terraform Graph

STEP – 7: DESTROY CLOUD RESOURCES

- To Destroy the cloud resources run the command terraform destroy.
- Terminate the all resources created by terraform

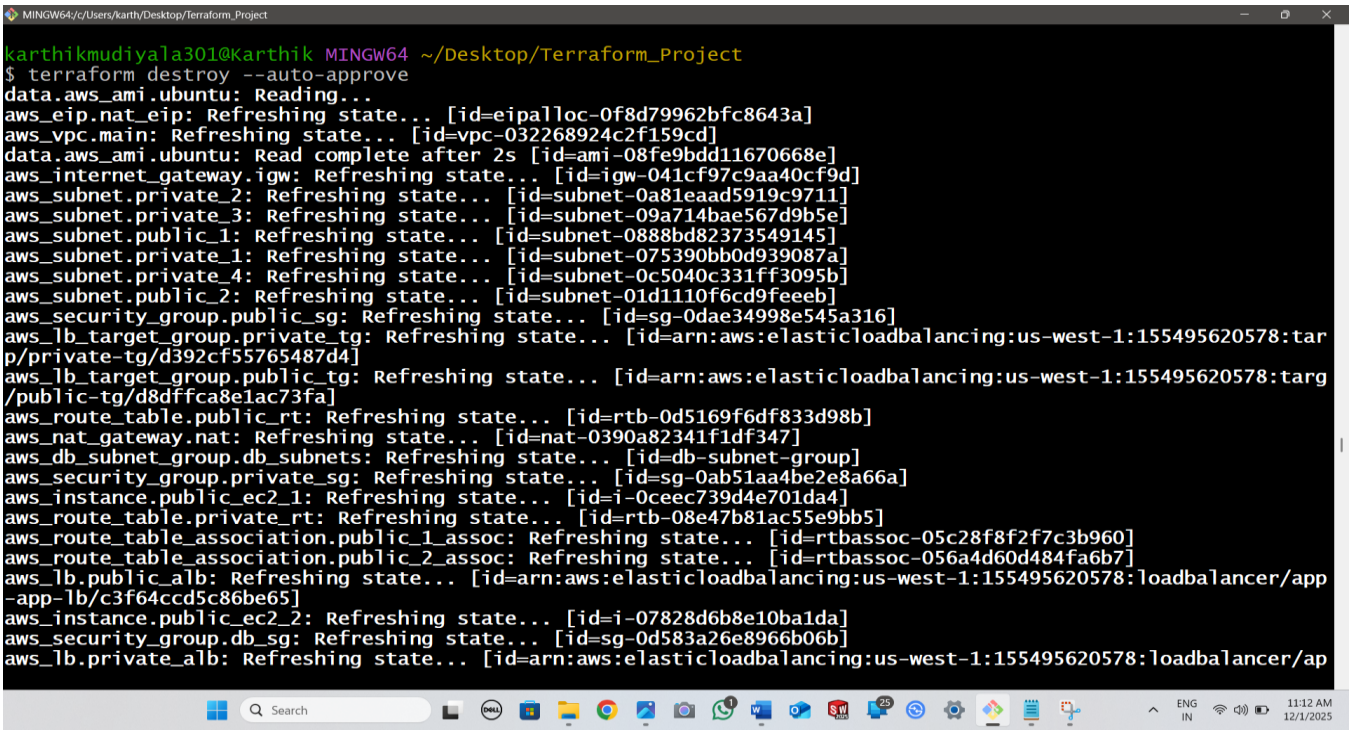


Fig-31: Destroy all resources create using terraform

CONCLUSION

This project successfully provisions a complete, production-grade cloud architecture on AWS using Terraform as the Infrastructure-as-Code (IaC) tool. The environment includes a fully functional VPC, multi-tier subnet design, secure networking, load balancing, and a highly available database layer.

The architecture begins with a custom VPC containing public and private subnets across multiple availability zones to ensure high availability. An Internet Gateway enables inbound and outbound internet access for resources in the public subnets, while a NAT Gateway securely allows private subnets to access the internet without exposing internal instances.

Multiple EC2 instances are deployed: two in public subnets (behind a **public Application Load Balancer**) to handle external traffic, and two in private subnets (behind a **private/internal Load Balancer**) to run application-tier workloads without public exposure. Security Groups enforce strict access control between layers.

For data persistence, the project deploys an RDS MySQL database in private subnets along with a Read Replica for improved performance and redundancy. Automated backups are enabled to support replication and recovery.

Terraform manages the entire lifecycle — creation, modification, and destruction — using reusable code, dependency handling, and state management. This ensures consistent, repeatable deployments with minimal manual effort.

Overall, the project demonstrates end-to-end infrastructure automation, secure network design, load-balanced compute tiers, and a scalable database architecture — meeting real-world cloud engineering and DevOps standards.